제4부 코드

제1부의 기법을 구현한 파이썬 모듈이다(Python 3, NumPy, SciPy, SymPy, Matplotlib). 모듈은 서로를 이름으로 가져오므로 한 폴더에 두고 쓴다. 판정과 국소화의 근거가 되는 부등식은 모두 이 코드 안에 있다.

core.py — 구간 산술, 시간 다항식(배정밀도·확장 정밀도), 번스타인 범위

```
"""core: interval arithmetic with outward padding (I), polynomials in the local time s whose coefficients are arrays over
the grid (P; float64 or long double; Bernstein range bounds), and a cheap upper bound of a matrix 2-norm."""
import math
import numpy as np
import sympy as sp
from math import comb

U = 2.0 ** -53
PAD = 1e-14
sexp = lambda x: math.inf if (x != x or x > 700) else math.exp(x)


# ---------------------------------------------------------------- interval arithmetic (outward padding for rounding)
class I:
    def __init__(s, lo, hi=None):
        lo = np.asarray(lo, float); hi = lo if hi is None else np.asarray(hi, float)
        s.lo = lo - PAD * np.abs(lo) - 1e-300; s.hi = hi + PAD * np.abs(hi) + 1e-300
    @staticmethod
    def c(x): return x if isinstance(x, I) else I(x)
    def __add__(s, o): o = I.c(o); return I(s.lo + o.lo, s.hi + o.hi)
    __radd__ = __add__
    def __neg__(s): return I(-s.hi, -s.lo)
    def __sub__(s, o): return s + (-I.c(o))
    def __rsub__(s, o): return I.c(o) + (-s)
    def __mul__(s, o):
        o = I.c(o); p = np.stack([s.lo * o.lo, s.lo * o.hi, s.hi * o.lo, s.hi * o.hi])
        return I(np.nanmin(p, 0), np.nanmax(p, 0))
    __rmul__ = __mul__
    def __truediv__(s, o):
        o = I.c(o); bad = (o.lo <= 0) & (o.hi >= 0)
        inv = I(np.where(bad, -np.inf, 1 / np.where(bad, 1, o.hi)), np.where(bad, np.inf, 1 / np.where(bad, 1, o.lo)))
        return s * inv
    def __rtruediv__(s, o): return I.c(o) / s
    def __pow__(s, n):
        n = int(n)
        if n < 0: return I(np.ones_like(s.lo)) / (s ** (-n))
        if n == 0: return I(np.ones_like(s.lo))
        a, b = s.lo ** n, s.hi ** n
        if n % 2: return I(a, b)
        lo = np.where((s.lo <= 0) & (s.hi >= 0), 0.0, np.minimum(a, b)); return I(lo, np.maximum(a, b))
    def exp(s): return I(np.exp(s.lo), np.exp(s.hi))
    def mag(s): return np.maximum(np.abs(s.lo), np.abs(s.hi))


def gexp(x):
    return x.exp() if isinstance(x, I) else np.exp(x)


# ---------------------------------------------------------------- polynomials in s whose coefficients are arrays over grid points
class P:
    def __init__(s, c):                                      # shape (deg+1, N); float64, or long double if given
        c = np.asarray(c); s.c = c if c.dtype == np.longdouble else c.astype(float)
    @staticmethod
    def k(x, N): return x if isinstance(x, P) else P(np.asarray(x, float).reshape(1, -1) * np.ones((1, N)))
    def __add__(s, o):
        o = P.k(o, s.c.shape[1]); n = max(len(s.c), len(o.c)); c = np.zeros((n, s.c.shape[1]), dtype=np.result_type(s.c, o.c)); c[:len(s.c)] += s.c; c[:len(o.c)] += o.c
        return P(c)
    __radd__ = __add__
    def __neg__(s): return P(-s.c)
    def __sub__(s, o): return s + (-P.k(o, s.c.shape[1]))
```

```
    def __rsub__(s, o): return P.k(o, s.c.shape[1]) + (-s)
    def __mul__(s, o):
        if not isinstance(o, P): return P(s.c * np.asarray(o, float))
        c = np.zeros((len(s.c) + len(o.c) - 1, s.c.shape[1]), dtype=np.result_type(s.c, o.c))
        for i in range(len(s.c)):
            for j in range(len(o.c)):
                c[i + j] += s.c[i] * o.c[j]
        return P(c)
    __rmul__ = __mul__
    def __pow__(s, n):
        if int(n) != n or n < 0: raise TypeError("rational")
        out = P(np.ones((1, s.c.shape[1]), dtype=s.c.dtype))
        for _ in range(int(n)): out = out * s
        return out
    def __truediv__(s, o): raise TypeError("rational")
    def __rtruediv__(s, o): raise TypeError("rational")
    def exp(s): raise TypeError("transcendental")
    def der(s): return P(np.array([j * s.c[j] for j in range(1, len(s.c))])) if len(s.c) > 1 else P(np.z
eros_like(s.c))
    def shift(s, s0):
        n = len(s.c); c = np.zeros_like(s.c)
        for j in range(n):
            for i in range(j, n):
                c[j] += comb(i, j) * s0 ** (i - j) * s.c[i]
        return P(c)
    def bern(s, h):
        m = len(s.c) - 1
        if s.c.dtype == np.longdouble:                          # extended precision: exact-ratio Bernste
in weights in long double
            hl = np.longdouble(h); a = [s.c[j] * hl ** j for j in range(m + 1)]
            b = np.array([sum(np.longdouble(comb(k, j)) / np.longdouble(comb(m, j)) * a[j] for j in rang
e(k + 1)) for k in range(m + 1)])
        else:
            a = [s.c[j] * h ** j for j in range(m + 1)]
            b = np.array([sum(comb(k, j) / comb(m, j) * a[j] for j in range(k + 1)) for k in range(m + 1
)])
        tol = 1e-13 * np.sum(np.abs(np.array(a)), axis=0) + 1e-300
        return b.min(0) - tol, b.max(0) + tol


def gexpP(x):
    if isinstance(x, P): raise TypeError("transcendental")
    return gexp(x)


def neumann_L(n, hx):
    if n == 1: return np.zeros((1, 1))
    L = np.diag(-2.0 * np.ones(n)) + np.diag(np.ones(n - 1), 1) + np.diag(np.ones(n - 1), -1)
    L[0, 0] = L[-1, -1] = -1.0
    return L / hx ** 2


def pnorm(A):
    return min(float(np.linalg.norm(A, "fro")), math.sqrt(float(np.abs(A).sum(0).max()) * float(np.abs(A
).sum(1).max())))
```

rig.py — 반올림 보조정리 R1–R3: 노름 상계, 엄밀 행렬 지수, 최대 고윳값 상계, 곱의 오차

```
"""rig: rounding-aware bounds for the certificate (standard floating-point model, round to nearest, u =
2^-53).
Every function returns an UPPER bound of the exact quantity it names; the matrix the bound refers to is
the stored
floating-point matrix (an exactly defined object)."""
import math
import numpy as np

U = 2.0 ** -53


def gam(k):
    k = float(k)
    return k * U / (1.0 - k * U)


def nb(M):
    """upper bound of ||M||_2 and of || |M| ||_2:  sqrt(||M||_1 ||M||_inf), with the rounding of the sum
s padded"""
    M = np.atleast_2d(M); n = max(M.shape)
    a = float(np.abs(M).sum(axis=0).max()); b = float(np.abs(M).sum(axis=1).max())
```

```
    return math.sqrt(a * b) * (1.0 + gam(n + 4))


def pnorm_up(M):
    """upper bound of ||M||_2 for the stored matrix: min(Frobenius, sqrt(||.||_1 ||.||_inf)), padded"""
    n = M.size
    f = float(np.linalg.norm(M, "fro")) * (1.0 + gam(n + 4))
    return min(f, nb(M))


def expm_rig(A, K=14):
    """X, r with ||exp(A) - X||_2 <= r.  Scaling by a power of two (exact), Taylor-Horner of degree K, squaring;
    truncation and rounding are bounded in the 2-norm with nb()."""
    n = A.shape[0]; I_ = np.eye(n)
    b = nb(A); s = 0
    while b / 2 ** s > 0.25: s += 1
    B = A / 2.0 ** s; bB = b / 2.0 ** s                      # exact scaling
    X = I_.copy(); e = 0.0
    for j in range(K, 0, -1):                                # X <- I + B X / j
        BX = B @ X; Xn = I_ + BX / j
        e = (bB * e + gam(n + 3) * bB * nb(X) + U * nb(Xn)) / 1.0 if j == K else (bB * e) / j + gam(n + 3) * (bB * nb(X) / j + nb(Xn))
        X = Xn
    trunc = bB ** (K + 1) / math.factorial(K + 1) / (1.0 - bB / (K + 2))
    e += trunc
    for _ in range(s):                                       # squaring: ||Y - fl(X^2)|| <= (2||X|| + e) e + gam ||X||^2
        nX = nb(X); X = X @ X
        e = (2.0 * nX + e) * e + gam(n + 2) * nX * nX
    return X, e


def lmax_sym_rig(S):
    """upper bound of lambda_max of the symmetric matrix S (stored floats).  Rump-type bound: with V, lam from eigh,
    R = S V - V diag(lam), G = V^T V - I; if ||G|| < 1, every eigenvalue is within ||R||/sqrt(1-||G||) of some lam_i.
    The smaller of this and the Gershgorin bound is returned."""
    n = S.shape[0]
    gersh = float(np.max(np.diag(S) + (np.abs(S).sum(axis=1) - np.abs(np.diag(S))))) + gam(n + 2) * float(np.abs(S).sum(axis=1).max())
    try:
        lam, V = np.linalg.eigh(S)
    except np.linalg.LinAlgError:
        return gersh
    R = S @ V - V * lam[None, :]; G = V.T @ V - np.eye(n)
    nV = nb(V); eR = nb(R) + gam(n + 2) * (nb(S) * nV + nV * float(np.max(np.abs(lam)))); eG = nb(G) + gam(n + 2) * nV * nV
    if eG >= 0.5: return gersh
    return min(gersh, float(lam.max()) + eR / math.sqrt(1.0 - eG) + gam(4) * abs(float(lam.max())))


def mul_rig(Xa, ea, Xb, eb, Na=None, Nb=None):
    """product of two enclosed matrices: exact Ta, Tb with ||Ta - Xa|| <= ea, ||Tb - Xb|| <= eb.
    Ta Tb - fl(Xa Xb) = Ta (Tb - Xb) + (Ta - Xa) Xb + (Xa Xb - fl(Xa Xb)), hence
        e <= Na * eb + ea * ||Xb|| + gam(n+2) || |Xa| || || |Xb| ||,   Na >= ||Ta|| (rigorous bound of the exact factor).
    Passing Na = e^{h mu} (log-norm bound) keeps the error from compounding with a crude norm of every factor."""
    n = Xa.shape[0]
    if Na is None: Na = pnorm_up(Xa) + ea
    return Xa @ Xb, Na * eb + ea * pnorm_up(Xb) + gam(n + 2) * nb(Xa) * nb(Xb)
```

operators.py — 경로 A의 입력: 공간 연산자 FDM·FVM·FEM(P1·P2·2차원)·DG, 로빈 경계, 일관성 점검

```
"""operators: spatial discretizations that turn a PDE into the semi-discrete system  u' = D (L u) + R(u) + s.
Every operator returns (L, w, x): the matrix L, the node weights w (the norm is ||e||_W = ||sqrt(w) e||_2, so that the
discrete inner product approximates the L2 inner product) and the node positions x.

  1D, bounded interval [0, Lx], natural (zero-flux) ends unless a Robin term is added:
    fdm_vertex(N, Lx)            second-order FDM on N vertices, ghost-point Neumann           (w: trapezoid weights)
    fvm_cell(N, Lx)              cell-centred finite volumes, two-point fluxes                  (w: cell widths)
```

```
 8      fem_p1(x)                    P1 finite elements on any node set, lumped mass                 (w: lu
    mped mass)
 9      fem_p1_graded(N, Lx, a)      fem_p1 on x = Lx (s - a sin(2 pi s)/(2 pi))  (spacing ratio (1-a):(1+a)
    , fine at both ends)
10      fem_p1_onesided(N, Lx, a)    fem_p1 on x = Lx (s - a sin(pi s)/pi)          (fine at x = 0)
11      dg_sipg(N, Lx, sigma)        symmetric interior-penalty DG, P1 per element, lumped (Lobatto) mass
12      robin_left(L, w, D, kap, c)  adds the boundary condition -D u_x(0) = kap (c - u(0)) in the weak (nat
    ural) form
13      robin_left_fvm(L, hx, D, kap, c)  the same condition as a boundary flux of the first cell
14    1D, periodic [0, 2 pi):
15      fdm4_periodic(N)             fourth-order central differences D1, D2 (circulant)
16      P1Periodic(N)                P1, consistent mass (circulant: Fourier modes are eigenvectors), advect
    ion symbol
17      P2Periodic(ndof, k16)        P2, consistent mass, group-FEM convection; k16 != 16 corrupts one eleme
    nt-stiffness entry
18    2D, periodic [0, 2 pi)^2:
19      P1Tri2D(N, a)                P1 triangles on a graded tensor mesh, lumped mass
20  Consistency checks (PDE spatial checklist, step 0):
21      pointwise_consistency(Lop, x, f, f2)   FDM/FVM: max |L f(x) - f''(x)| (stencil truncation error)
22      galerkin_consistency(P2, k)            FEM: max |M^-1 (K u_I - b)|, b_i = int (-u'') phi_i (weak res
    idual)"""
23  import math
24  import numpy as np
25  import scipy.sparse as sps
26  from scipy.sparse.linalg import splu
27
28
29  # ---------------------------------------------------------------------- 1D bounded interval
30  def fdm_vertex(N, Lx):
31      h = Lx / (N - 1); x = np.linspace(0, Lx, N)
32      L = (np.diag(-2.0 * np.ones(N)) + np.diag(np.ones(N - 1), 1) + np.diag(np.ones(N - 1), -1)) / h ** 2
33      L[0, 1] = L[-1, -2] = 2.0 / h ** 2                    # ghost point u_{-1} = u_1 (zero flux)
34      w = np.full(N, h); w[0] = w[-1] = h / 2               # trapezoid weights: w L is symmetric
35      return L, w, x
36
37
38  def fvm_cell(N, Lx):
39      h = Lx / N; x = h * (np.arange(N) + 0.5)
40      L = np.diag(-2.0 * np.ones(N)) + np.diag(np.ones(N - 1), 1) + np.diag(np.ones(N - 1), -1)
41      L[0, 0] = L[-1, -1] = -1.0                            # zero boundary flux
42      return L / h ** 2, h * np.ones(N), x
43
44
45  def fem_p1(x):
46      x = np.asarray(x, float); N = len(x); K = np.zeros((N, N)); w = np.zeros(N)
47      for e in range(N - 1):
48          l = x[e + 1] - x[e]; K[e:e + 2, e:e + 2] += np.array([[1, -1], [-1, 1]]) / l; w[e:e + 2] += l /
    2
49      return -K / w[:, None], w, x
50
51
52  def fem_p1_graded(N, Lx, a=0.5):
53      xi = np.linspace(0, 1, N); return fem_p1(Lx * (xi - a * np.sin(2 * np.pi * xi) / (2 * np.pi)))
54
55
56  def fem_p1_onesided(N, Lx, a=0.5):
57      s = np.linspace(0, 1, N); return fem_p1(Lx * (s - a * np.sin(np.pi * s) / np.pi))
58
59
60  def dg_sipg(N, Lx, sigma=4.0):
61      Ne = N // 2; he = Lx / Ne; A = np.zeros((N, N)); w = np.full(N, he / 2)
62      x = np.array([[e * he, (e + 1) * he] for e in range(Ne)]).ravel(); der = []
63      for e in range(Ne):
64          A[2 * e:2 * e + 2, 2 * e:2 * e + 2] += np.array([[1, -1], [-1, 1]]) / he
65          d = np.zeros(N); d[2 * e] = -1 / he; d[2 * e + 1] = 1 / he; der.append(d)
66      for e in range(Ne - 1):
67          jv = np.zeros(N); jv[2 * e + 1] = 1; jv[2 * e + 2] = -1; av = 0.5 * (der[e] + der[e + 1])
68          A += -(np.outer(jv, av) + np.outer(av, jv)) + sigma / he * np.outer(jv, jv)
69      if np.linalg.eigvalsh(A).min() < -1e-10: raise ValueError("SIPG penalty too small (not coercive)")
70      return -A / w[:, None], w, x
71
72
73  def robin_left(L, w, D, kap, c):
74      """weak form: D int u_x v_x + kap u(0) v(0) = kap c v(0) + ...  ->  D L[0,0] -= kap / w_0, source ka
    p c / w_0"""
75      L = L.copy(); L[0, 0] -= kap / (D * w[0]); return L, kap * c / w[0]
76
77
78  def robin_left_fvm(L, hx, D, kap, c):
79      """boundary flux of the first cell: (kap (c - u_0)) / hx, i.e. D L[0,0] -= kap/hx, source kap c / hx
```

```
    """
80      L = L.copy(); L[0, 0] -= kap / (hx * D); return L, kap * c / hx
81
82
83  # ---------------------------------------------------------------- 1D periodic
84  def fdm4_periodic(N):
85      h = 2 * np.pi / N; x = h * np.arange(N)
86      D1 = lambda f: (-np.roll(f, -2) + 8 * np.roll(f, -1) - 8 * np.roll(f, 1) + np.roll(f, 2)) / (12 * h)
87      D2 = lambda u, c16=16.0: (-np.roll(u, -2) + c16 * np.roll(u, -1) - 30 * u + c16 * np.roll(u, 1) - np
    .roll(u, 2)) / (12 * h * h)
88      k = np.fft.fftfreq(N, 1.0 / N); omega = (8 * np.sin(k * h) - np.sin(2 * k * h)) / (6 * h)   # D1 e^{
    ikx} = i omega e^{ikx}
89      return D1, D2, x, omega
90
91
92  class P1Periodic:
93      """uniform P1, consistent mass.  Symbols: M -> h (4 + 2 cos kh)/6, C -> i sin kh, so M^-1 C e^{ikx}
    = i omega_k e^{ikx},
94      omega_k = 3 sin(kh) / (h (2 + cos kh))."""
95      def __init__(s, N):
96          s.n = N; h = 2 * np.pi / N; s.h = h; s.x = h * np.arange(N); k = np.fft.fftfreq(N, 1.0 / N); s.k
     = k
97          s.msym = h * (4 + 2 * np.cos(k * h)) / 6; s.omega = 3 * np.sin(k * h) / (h * (2 + np.cos(k * h))
    )
98
99      def adv(s, u):                                      # -M^-1 C u  (the semi-discrete -u_x)
100         Cu = 0.5 * (np.roll(u, -1) - np.roll(u, 1)); return -np.real(np.fft.ifft(np.fft.fft(Cu) / s.msym
    ))
101
102     def weight_sqrt(s):                                 # M^{1/2} as a dense circulant matrix
103         return np.real(np.fft.ifft(np.diag(np.sqrt(s.msym)) @ np.fft.fft(np.eye(s.n), axis=0), axis=0))
104
105
106 MP2 = np.array([[4, 2, -1], [2, 16, 2], [-1, 2, 4]], float) / 30.0
107 KP2 = np.array([[7, -8, 1], [-8, 16, -8], [1, -8, 7]], float) / 3.0
108 CP2 = np.array([[-0.5, 2 / 3, -1 / 6], [-2 / 3, 0, 2 / 3], [1 / 6, -2 / 3, 0.5]])
109
110
111 class P2Periodic:
112     """P2 on a uniform periodic mesh, consistent mass (sparse LU); DOF order vertex, midpoint, vertex, .
    ..
113     diff(u) = M^-1(-nu K u), conv(f) = M^-1(-C f) with C_ij = int phi_i phi_j' (group FEM for a flux f).
    """
114     def __init__(s, ndof, nu=0.0, k16=16.0):
115         Ne = ndof // 2; s.Ne, s.n, s.nu = Ne, 2 * Ne, nu; h = 2 * np.pi / Ne; s.h = h; s.x = (h / 2) * n
    p.arange(2 * Ne)
116         Ke = KP2.copy(); Ke[1, 1] = k16 / 3.0
117         M = sps.lil_matrix((s.n, s.n)); K = sps.lil_matrix((s.n, s.n)); C = sps.lil_matrix((s.n, s.n))
118         for e in range(Ne):
119             idx = [2 * e, 2 * e + 1, (2 * e + 2) % s.n]
120             for a in range(3):
121                 for b in range(3):
122                     M[idx[a], idx[b]] += MP2[a, b] * h; K[idx[a], idx[b]] += Ke[a, b] / h; C[idx[a], idx
    [b]] += CP2[a, b]
123         s.M, s.K, s.C = M.tocsc(), K.tocsr(), C.tocsr(); s.lu = splu(s.M); s.mass = np.asarray(s.M.sum(a
    xis=0)).ravel()
124
125     def Minv(s, v): return s.lu.solve(v)
126     def diff(s, u): return s.nu * s.Minv(-(s.K @ u))
127     def conv(s, f): return s.Minv(-(s.C @ f))
128     def dense(s, op):
129         I_ = np.eye(s.n); return np.column_stack([op(I_[:, j]) for j in range(s.n)])
130
131
132 class P1Tri2D:
133     """periodic P1 triangles on x_i = s_i + a sin s_i (both directions), each quad split along one diago
    nal; lumped mass."""
134     def __init__(s, N, a=0.3):
135         s1 = 2 * np.pi * np.arange(N) / N; xs = s1 + a * np.sin(s1); s.N = N
136         s.X, s.Y = np.meshgrid(xs, xs, indexing="ij"); hx = np.diff(np.append(xs, 2 * np.pi))
137         n = N * N; idx = lambda i, j: (i % N) * N + (j % N); K = sps.lil_matrix((n, n)); m = np.zeros(n)
138         for i in range(N):
139             for j in range(N):
140                 p00, p10, p01, p11 = np.array([0.0, 0.0]), np.array([hx[i], 0.0]), np.array([0.0, hx[j]]
    ), np.array([hx[i], hx[j]])
141                 for tri, ids in (((p00, p10, p11), (idx(i, j), idx(i + 1, j), idx(i + 1, j + 1))),
142                                  ((p00, p11, p01), (idx(i, j), idx(i + 1, j + 1), idx(i, j + 1)))):
143                     Pm = np.array(tri); B = np.array([Pm[1] - Pm[0], Pm[2] - Pm[0]]).T; area = 0.5 * abs
    (np.linalg.det(B))
144                     G = np.linalg.inv(B).T @ np.array([[-1, 1, 0], [-1, 0, 1]], float); Ke = area * G.T
```

```
    @ G
                    for a_ in range(3):
                        m[ids[a_]] += area / 3
                        for b_ in range(3): K[ids[a_], ids[b_]] += Ke[a_, b_]
        s.K = K.tocsr(); s.m = m

    def lap(s, u): return -(s.K @ u.ravel() / s.m).reshape(u.shape)


# ------------------------------------------------------------------ consistency checks (diagnostics of the PDE checklist)
def pointwise_consistency(Lop, x, f, f2):
    """FDM / FVM: stencil truncation error max |L f(x_i) - f''(x_i)| (call on a sequence of grids, read the slope)"""
    return float(np.max(np.abs(Lop(f(x)) - f2(x))))


def galerkin_consistency(F, k=3):
    """FEM (P2Periodic): weak residual max |M^-1 (K u_I - b)| for u = sin(kx), b_i = int k^2 sin(kx) phi_i (5-point Gauss).
    The pointwise residual M^-1 K u_I + (u'')_I does not reflect the convergence order of a Galerkin method."""
    gx, gw = np.polynomial.legendre.leggauss(5); xi = 0.5 * (gx + 1); gw = 0.5 * gw
    phi = np.array([2 * (xi - 0.5) * (xi - 1), 4 * xi * (1 - xi), 2 * xi * (xi - 0.5)]); b = np.zeros(F.n)
    for e in range(F.Ne):
        xq = F.h * (e + xi); fq = k * k * np.sin(k * xq); ids = [2 * e, 2 * e + 1, (2 * e + 2) % F.n]
        for a in range(3): b[ids[a]] += F.h * np.sum(gw * fq * phi[a])
    return float(np.max(np.abs(F.Minv(F.K @ np.sin(k * F.x) - b))))
```

semidisc.py — 경로 A의 계와 후보(시간 스킴 출력 + 에르미트, 테일러, 사영, 신경망)

```
"""semidisc: the semi-discrete system and its candidates.
    Sys(spec)        spec = dict(m, N, L, w, D, T, u0, R[, src]): u' = D_a (L u_a) + R_a(u) + src, R a polynomial list in the
                     m species (sympy-able); builds F, the Jacobian, the Hessian, polynomial and interval evaluators and the
                     absolute-coefficient polynomial (for rounding bounds).
Candidates are lists of coefficient arrays C[k] (K+1, mN) in the local time s of step k (one polynomial per step):
    cand_taylor(S, n, K)   Taylor-K recursion (Leibniz rule on the polynomial right-hand side)
    cand_rk4h(S, n)        RK4 values joined by cubic Hermite interpolation (the output of a user's code)
    cand_from_values(S, t, Y)  ANY time scheme: its step times t (variable steps allowed) and values Y joined by cubic
                           Hermite interpolation with slopes F(Y); returns (C, hk) for certify(S, C, hk=hk)
    project(fun, T, n, K)  degree-K Lobatto projection of any function of t (used for PINN and other surrogates)
    cand_pinn(S, n, K)     windowed tanh network trained by Levenberg-Marquardt on the residual, then projected
    true_error(S, C)       distance at T to an independent integrator (diagnostic only)"""
import math
import numpy as np
import sympy as sp
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares
from core import I, P, U, pnorm


# ------------------------------------------------------------------ system
class Sys:
    def __init__(s, spec):
        s.spec = spec; s.m, s.N, s.D = spec["m"], spec["N"], spec["D"]
        s.L = spec["L"]; s.w = spec["w"]; s.u0 = np.asarray(spec["u0"], float).ravel(); s.T = spec["T"]
        m = s.m; syms = sp.symbols(f"s0:{m}"); Rs = [sp.sympify(e) for e in spec["R"](list(syms))]
        s.Jf = [[sp.lambdify(syms, sp.diff(Rs[a], syms[b]), modules="math") for b in range(m)] for a in range(m)]
        s.Jnp = [[sp.lambdify(syms, sp.diff(Rs[a], syms[b]), modules="numpy") for b in range(m)] for a in range(m)]
        s.Hf = [[[sp.lambdify(syms, sp.diff(Rs[a], syms[b], syms[c]), modules="math") for c in range(m)] for b in range(m)] for a in range(m)]
        s.R_np = sp.lambdify(syms, Rs, modules="numpy"); s.R_gen = sp.lambdify(syms, Rs, modules="math")
        # majorant of the EVALUATION TREE of R (the tree lambdify evaluates): every sum becomes a sum of majorants, every
        # product a product of majorants, every constant its absolute value; at |u| it bounds every partial result of
        # the evaluation, so gam(#operations) * majorant bounds its rounding (and the expanded absolute polynomial,
```

```
        # which can be smaller when the tree has internal cancellation, is never used)
        s.Rabs_gen = sp.lambdify(syms, [_majorant(e_) for e_ in Rs], modules="math")
        s.R_ops = max([_opcount(e_) for e_ in Rs] + [4])
        s.Jnz = [(a, b) for a in range(m) for b in range(m) if sp.diff(Rs[a], syms[b]) != 0]          # structural nonzeros
        s.Hnz = [(a, b, c) for a in range(m) for b in range(m) for c in range(m) if sp.diff(Rs[a], syms[b], syms[c]) != 0]
        s.sw = np.tile(np.sqrt(s.w), m)
        s.src = np.asarray(spec.get("src", np.zeros(m * s.N)), float).ravel()     # constant source (e.g. Robin supply)

    def F(s, z):
        u = z.reshape(s.m, s.N); r = s.R_np(*u)
        return np.concatenate([s.D[a] * (s.L @ u[a]) + np.asarray(r[a], float) * np.ones(s.N) for a in range(s.m)]) + s.src

    def Jnum(s, u):
        m, N = s.m, s.N; J = np.zeros((m * N, m * N))
        for a in range(m):
            for b in range(m):
                v = np.asarray(s.Jnp[a][b](*u), float) * np.ones(N)
                J[a * N:(a + 1) * N, b * N:(b + 1) * N] = np.diag(v) + (s.D[a] * s.L if a == b else 0)
        return J

    def Rlist(s, args, f):
        out = f(*args); return [o if isinstance(o, (I, P)) else np.asarray(o, float) * np.ones(s.N) for o in out]


# ------------------------------------------------------------------ candidates: list of coefficient arrays (deg+1, mN) in local s
def cand_rk4h(S, n):
    h = S.T / n; y = S.u0.copy(); C = []
    for _ in range(n):
        k1 = S.F(y); k2 = S.F(y + h / 2 * k1); k3 = S.F(y + h / 2 * k2); k4 = S.F(y + h * k3)
        b = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4); fa, fb = k1, S.F(b); dl = b - y
        C.append(np.array([y, fa, 3 * dl / h ** 2 - (2 * fa + fb) / h, -2 * dl / h ** 3 + (fa + fb) / h ** 2])); y = b
    return C


def cand_taylor(S, n, K=8):
    h = S.T / n; y = S.u0.copy(); C = []; m, N = S.m, S.N
    for _ in range(n):
        c = [y.copy()]
        for j in range(K):
            Ps = [P(np.array(c)[:, a * N:(a + 1) * N]) for a in range(m)]
            r = [P.k(v, N) for v in S.R_gen(*Ps)]
            nxt = np.concatenate([(S.D[a] * (S.L @ c[j][a * N:(a + 1) * N]) + (r[a].c[j] if len(r[a].c) > j else 0)
                                   + (S.src[a * N:(a + 1) * N] if j == 0 else 0)) / (j + 1) for a in range(m)])
            c.append(nxt)
        c = np.array(c); C.append(c)
        y = c[-1].copy()
        for j in range(K - 1, -1, -1): y = c[j] + h * y                  # Horner p(h)
    return C


LOB = lambda K: 0.5 * (1 - np.cos(np.pi * np.arange(K + 1) / K))          # Lobatto nodes on [0,1]


def project(fun, T, n, K=8):
    h = T / n; sig = LOB(K); V = np.vander(sig, K + 1, increasing=True); Vi = np.linalg.inv(V); C = []
    for k in range(n):
        vals = np.array([fun(k * h + s * h) for s in sig])               # (K+1, mN)
        a = Vi @ vals; C.append(a / (h ** np.arange(K + 1))[:, None])
    return C


def train_pinn(S, Tw=None, H=24, ncol=240, seed=0):
    """windowed PINN for an ODE (N=1): u(t) = u_s + (t-t_s) * (W2 tanh(W1 t~ + b1) + b2), t~ in [-1,1]; LM on the residual."""
    assert S.N == 1; m = S.m; rng = np.random.default_rng(seed); Tw = Tw or S.T / 20
    nw = int(round(S.T / Tw)); nets = []; us = S.u0.copy()
    for wdx in range(nw):
        ts = wdx * Tw; tc = np.linspace(-1, 1, ncol)
        def unpack(th):
            W1 = th[:H]; b1 = th[H:2 * H]; W2 = th[2 * H:2 * H + m * H].reshape(m, H); b2 = th[2 * H + m * H:]
```

```
            return W1, b1, W2, b2
        def ev(th, tt):
            W1, b1, W2, b2 = unpack(th); a = np.tanh(np.outer(tt, W1) + b1); net = a @ W2.T + b2
            dnet = ((1 - a ** 2) * W1) @ W2.T * (2 / Tw)
            tau = (tt + 1) * Tw / 2; u = us + tau[:, None] * net; du = net + tau[:, None] * dnet
            return u, du
        def res(th):
            u, du = ev(th, tc); f = np.array([S.F(ui) for ui in u]); return (du - f).ravel()
        def jac(th):
            W1, b1, W2, b2 = unpack(th); z = np.outer(tc, W1) + b1; a = np.tanh(z); sp_ = 1 - a ** 2
            tau = (tc + 1) * Tw / 2; net = a @ W2.T + b2; u = us + tau[:, None] * net
            nP = len(th); dnet = np.zeros((ncol, m, nP)); ddn = np.zeros((ncol, m, nP)); k2 = 2 / Tw
            for i in range(m):
                dnet[:, i, 2 * H + i * H:2 * H + (i + 1) * H] = a
                dnet[:, i, 2 * H + m * H + i] = 1
                ddn[:, i, 2 * H + i * H:2 * H + (i + 1) * H] = k2 * sp_ * W1
            dnet[:, :, :H] = (W2[None, :, :] * (sp_ * tc[:, None])[:, None, :])
            dnet[:, :, H:2 * H] = (W2[None, :, :] * sp_[:, None, :])
            ddn[:, :, :H] = k2 * W2[None, :, :] * (sp_ + W1 * (-2 * a * sp_) * tc[:, None])[:, None, :]
            ddn[:, :, H:2 * H] = k2 * W2[None, :, :] * (W1 * (-2 * a * sp_))[:, None, :]
            du_ = dnet + tau[:, None, None] * ddn; du_u = tau[:, None, None] * dnet
            Jr = np.array([S.Jnum(ui.reshape(m, 1)) for ui in u])          # (ncol, m, m)
            return (du_ - np.einsum("pij,pjk->pik", Jr, du_u)).reshape(ncol * m, nP)
        best = None
        for trial in range(1):
            th0 = 0.5 * rng.standard_normal(2 * H + m * H + m)
            r = least_squares(res, th0, jac=jac, method="lm", xtol=1e-15, ftol=1e-15, gtol=1e-15, max_nf
ev=1500)
            if best is None or r.cost < best.cost: best = r
        th = best.x; nets.append((ts, us.copy(), th))
        us = ev(th, np.array([1.0]))[0][0]
    def fun(t):
        wdx = min(int(t / Tw), nw - 1); ts, u_s, th = nets[wdx]
        W1, b1, W2, b2 = th[:H], th[H:2 * H], th[2 * H:2 * H + m * H].reshape(m, H), th[2 * H + m * H:]
        tt = 2 * (t - ts) / Tw - 1; net = W2 @ np.tanh(W1 * tt + b1) + b2
        return u_s + (t - ts) * net
    return fun, nets


def cand_pinn(S, n, K=8):
    fun, nets = train_pinn(S)
    return project(fun, S.T, n, K)


# ---------------------------------------------------------------------- certification
def true_error(S, C, hk=None):
    n = len(C); h = S.T / n if hk is None else float(hk[-1]); Ck = C[-1]; pend = Ck[-1].copy()
    for j in range(Ck.shape[0] - 2, -1, -1): pend = Ck[j] + h * pend
    ref = solve_ivp(lambda t, z: S.F(z), (0, S.T), S.u0, method="DOP853", rtol=1e-13, atol=1e-15).y[:, -
1]
    sw = getattr(S, "sw_used", S.sw)
    return float(np.linalg.norm(sw * (pend - ref))), float(np.linalg.norm(S.sw * (pend - ref)))

def cand_from_values(S, t, Y, dY=None):
    """candidate from the output of any time scheme (one-step, multistep, implicit, adaptive, splitting,
 ...):
    step times t[0..n] (t[0] = 0, t[n] = T, any spacing) and values Y[0..n]; slopes F(Y) unless given.
    Returns (C, hk): C[k] the cubic Hermite polynomial on [t_k, t_k+1] in local s, hk the step sizes."""
    t = np.asarray(t, float); Y = np.asarray(Y, float); hk = np.diff(t)
    if abs(t[0]) > 0 or abs(t[-1] - S.T) > 1e-12 * max(1.0, S.T): raise ValueError("steps must cover [0,
 T]")
    Fv = [S.F(y) for y in Y] if dY is None else list(dY); C = []
    for k, h in enumerate(hk):
        y0, y1, f0, f1 = Y[k], Y[k + 1], Fv[k], Fv[k + 1]; dl = y1 - y0
        C.append(np.array([y0, f0, (3 * dl / h - 2 * f0 - f1) / h, (f0 + f1 - 2 * dl / h) / h ** 2]))
    return C, hk


def _majorant(e):
    """majorant expression of a polynomial expression tree (inputs are the absolute values of the variab
les)"""
    if e.is_Symbol: return e
    if e.is_Number: return abs(e)
    if e.is_Add: return sp.Add(*[_majorant(a) for a in e.args], evaluate=False)
    if e.is_Mul: return sp.Mul(*[_majorant(a) for a in e.args], evaluate=False)
    if e.is_Pow and e.exp.is_Integer and e.exp >= 0: return sp.Pow(_majorant(e.base), e.exp, evaluate=Fa
lse)
    raise ValueError("R must be a polynomial expression")


```

```
def _opcount(e):
    """number of floating-point operations in evaluating the tree (an integer power n counts as n - 1 pr
oducts)"""
    if e.is_Symbol or e.is_Number: return 0
    if e.is_Add or e.is_Mul: return len(e.args) - 1 + sum(_opcount(a) for a in e.args)
    if e.is_Pow: return max(int(e.exp) - 1, 1) + _opcount(e.base)
    return 8
```

cert.py — 경로 A: 동결 전파자 증명서, 사후 고정점, 인과적 닫기, 처방, 망원 항등식, 국소화

```
"""cert: the boolean certificate for a candidate of u' = D (L u) + R(u) + s (R polynomial).
    proved(P(candidate, [0, t], eps)) is True only when the post-fixed point L(Delta*) <= Delta* closes,
 where L is written
    with integrals (defect, Bernstein bounds) and discrete chain products (frozen propagators) only.
  step_data   per step: q sub-steps, frozen Jacobian at sub-step midpoints, freezing error and Hessian r
emainder on Bernstein
              boxes, defect norm (coefficients in long double), log-norm upper bound, rigorous expm of e
ach sub-step
  make_ctx    time-local species scaling (the weight change is carried exactly inside the step matrix)
  Cert        products of the stored step matrices (telescoping form: rounding enters as level-proportio
nal forcing),
              checkpoint blocks, causal prefix closure, interior windows, attribution by step and by ter
m
  certify     prescription rounds (cap lowering, q x 4, local Taylor replacement), returns the history a
nd the best round
  certify_best  both species scalings (time-local and global), keep the longer proved interval
  hk            optional step sizes: variable (adaptive) steps are allowed; the candidate's step k lives
 on [t_k, t_k + hk[k]]
  Cert.localize  where and how: the bound split exactly by step and term, each term placed on (component
, node)
Rounding: every bound holds for the exact quantities (standard floating-point model; see rig.py)."""
import json, math, time
import numpy as np
from scipy.linalg import expm
from core import I, P, U, pnorm
from rig import gam, nb, pnorm_up, expm_rig, lmax_sym_rig, mul_rig
GAM64 = lambda k: k * 2.0 ** -64 / (1 - k * 2.0 ** -64)          # standard model for x87 extended preci
sion
RIG = True
PADF = 1.0 + 1e-10         # final pad on scalar sums of positive terms (covers gam(n) for n < 4e5)


def step_data(S, C, k, q, ctx, h=None):
    """everything the chain product needs from step k: sub-step freezing error, Hessian coefficient, def
ect norm,
    log-norm, frozen step propagator (and inverse), end-of-step jump"""
    m, N, D, L = S.m, S.N, S.D, S.L; n = len(C); h = S.T / n if h is None else float(h); hs = h / q
    cx = ctx["steps"][k]; sig, sw, swmin_n, dcap = cx["sig"], cx["sw"], cx["swmin_n"], cx["dcap"]
    sw_next = ctx["steps"][k + 1]["sw"] if k + 1 < n else sw
    o_frm = np.zeros(q); o_hm = np.zeros(q); o_gn = np.zeros(q); o_mu = np.zeros(q)
    Ck = C[k]; deg = Ck.shape[0] - 1
    Ps = [P(Ck[:, a * N:(a + 1) * N]) for a in range(m)]
    lin = [Ps[a].der() - P(np.array([D[a] * (L @ Ps[a].c[j]) + (S.src[a * N:(a + 1) * N] if j == 0 else
0) for j in range(deg + 1)])) for a in range(m)]
    gfull = None
    try:
        if RIG:   # the defect coefficients are formed in long double (u = 2^-64): the cancellation betw
een large reaction
                  # terms then costs gam64(200) * (absolute-coefficient polynomial) instead of gam(200)
* (same)
            LD = np.longdouble; Pl = [P(Ck[:, a * N:(a + 1) * N].astype(LD)) for a in range(m)]; Lld = L
.astype(LD); srl = S.src.astype(LD)
            linl = [Pl[a].der() - P(np.array([LD(D[a]) * (Lld @ Pl[a].c[j]) + (srl[a * N:(a + 1) * N] if
 j == 0 else 0) for j in range(deg + 1)])) for a in range(m)]
            Rl = [P.k(r, N) if isinstance(r, P) else P(np.asarray(r, LD).reshape(1, -1) * np.ones((1, N)
, LD)) for r in S.R_gen(*Pl)]
            gl = np.concatenate([np.max(np.abs(np.array((linl[a] - Rl[a]).bern(h))), axis=0) for a in ra
nge(m)])
            gfull = gl.astype(float) * (1 + 2.0 ** -52)                       # round the long-double bo
und up to float64
            Pa_ = [P(np.abs(Pa.c)) for Pa in Ps]; Ra = [P.k(r, N) for r in S.Rabs_gen(*Pa_)]
            hp = lambda c: np.sum(np.abs(c) * (h ** np.arange(c.shape[0]))[:, None], 0)
            magd = np.concatenate([hp(Pa_[a].der().c if Pa_[a].c.shape[0] > 1 else np.zeros((1, N)))
                                   + np.abs(D[a]) * (np.abs(L) @ hp(Pa_[a].c)) + np.abs(S.src[a * N:(a +
 1) * N]) + hp(Ra[a].c)
                                   for a in range(m)])
            gfull = gfull + GAM64(200) * magd * (1 + 1e-10)
        else:
            Rp = [P.k(r, N) for r in S.R_gen(*Ps)]
```

```
            gfull = np.concatenate([np.max(np.abs(np.array((lin[a] - Rp[a]).bern(h))), axis=0) for a in range(m)])
    except TypeError:
        raise ValueError("R must be polynomial (rational / transcendental R: outside this pipeline)")
    Eb = np.eye(m * N)
    # all q sub-steps at once: Bernstein boxes and midpoints stacked along the grid axis (length q*N)
    subs = [[Pa.shift(j * hs) for j in range(q)] for Pa in Ps]
    bb = [[Sj.bern(hs) for Sj in sa] for sa in subs]
    shpad = [gam(64) * np.tile(np.sum(np.abs(Ps[a].c) * (h ** np.arange(Ps[a].c.shape[0]))[:, None], 0), q) for a in range(m)]
    box = [I(np.concatenate([x[0] for x in bb[a]]) - shpad[a], np.concatenate([x[1] for x in bb[a]]) + shpad[a]) for a in range(m)]
    midc = [[Sj.shift(hs / 2).c[0] for Sj in sa] for sa in subs]
    mid = [np.concatenate(midc[a]) for a in range(m)]
    fr = np.zeros(q * N)
    for a, b in S.Jnz:
        Jb = S.Jf[a][b](*box); Jb = Jb if isinstance(Jb, I) else I(np.asarray(Jb, float) * np.ones(q * N))
        jm = np.asarray(S.Jnp[a][b](*mid), float) * np.ones(q * N)
        fr += (sig[a] / sig[b] * np.maximum(np.abs(Jb.hi - jm), np.abs(Jb.lo - jm))) ** 2
    wide = [I(box[b].lo - dcap[b], box[b].hi + dcap[b]) for b in range(m)]
    sa = np.zeros((m, q * N))
    for a, b, c in S.Hnz:
        Hv = S.Hf[a][b][c](*wide); Hv = Hv if isinstance(Hv, I) else I(np.asarray(Hv, float) * np.ones(q * N))
        sa[a] += sig[a] / (sig[b] * sig[c]) * Hv.mag()
    rem = np.sum(sa ** 2, axis=0)
    fr_node = np.sqrt(fr).reshape(q, N).max(0); rem_node = np.sqrt(rem).reshape(q, N).max(0)
    o_frm = np.sqrt(fr).reshape(q, N).max(1); o_hm = 0.5 * np.sqrt(rem).reshape(q, N).max(1) / swmin_n
    o_gn[:] = float(np.linalg.norm(sw * gfull))
    eE = 0.0; Ei = np.eye(m * N); eEi = 0.0; nd = m * N
    for j in range(q):
        Jm = S.Jnum([midc[a][j] for a in range(m)])
        Jw = sw[:, None] * Jm / sw[None, :]
        if RIG:
            # the frozen matrix of the argument is the stored Jw; its distance to the exact weighted J_mid (formation of
            # D L, the weights and the division) is <= 3u |Jw|, added to the freezing error of this sub-step
            o_frm[j] += 3 * U * nb(Jw)
            Sym = 0.5 * (Jw + Jw.T); o_mu[j] = lmax_sym_rig(Sym) + U * nb(Sym)
            A = hs * Jw; Xa, ea = expm_rig(A); Xi, ei = expm_rig(-A)
            ea += U * nb(A) * math.exp(min(700, nb(A)))          # rounding of hs*Jw
            ei += U * nb(A) * math.exp(min(700, nb(A)))
            Eb, eE = mul_rig(Xa, ea, Eb, eE, Na=math.exp(min(700, hs * o_mu[j])))   # exact factor ||e^{hs Jw}|| <= e^{hs mu}
            Ei, eEi = mul_rig(Ei, eEi, Xi, ei)
        else:
            o_mu[j] = float(np.max(np.linalg.eigvalsh(0.5 * (Jw + Jw.T))))
            Eb = expm(hs * Jw) @ Eb
    pend = Ck[-1].copy()
    for j in range(deg - 1, -1, -1): pend = Ck[j] + h * pend
    nxt = C[k + 1][0] if k + 1 < n else pend
    mag = np.sum(np.abs(Ck) * (h ** np.arange(deg + 1))[:, None], 0)
    jvec = sw_next * (pend - nxt)
    o_jump = float(np.linalg.norm(sw_next * (pend - nxt))) + 2 * deg * U * float(np.linalg.norm(sw_next * mag))
    ratio = sw_next / sw; Eb = ratio[:, None] * Eb                 # E_k maps step-k weights to step-(k+1) weights
    if RIG:
        rmax = float(np.max(np.abs(ratio))); eE = eE * rmax + U * nb(Eb)
        Ei = Ei / ratio[None, :]; eEi = eEi / float(np.min(np.abs(ratio))) + U * nb(Ei)
        o_jump *= PADF
        return dict(frm=o_frm, hm=o_hm, gn=o_gn, mu=o_mu, jump=o_jump, E=Eb, Ei=Ei, eE=eE, eEi=eEi, conv=float(np.max(ratio)) * (1 + 2 * U),
                    where=dict(defect=(sw * gfull) ** 2, freezing=fr_node, quadratic=rem_node, jump=jvec ** 2))
    return dict(frm=o_frm, hm=o_hm, gn=o_gn, mu=o_mu, jump=o_jump, E=Eb, Ei=np.linalg.inv(Eb), conv=float(np.max(ratio)))


def make_ctx(S, C, cap_rel=1e-2, seg=200):
    """time-local species scaling: on each segment of `seg` steps, sigma_a = 1/max|p_a| over that segment (floor 1e-2).
    Weights change only at segment ends; the step propagator there carries the diagonal change of weights exactly."""
    m, N = S.m, S.N; n = len(C); steps = []; swmin_n = math.sqrt(float(np.min(S.w)))
    starts = np.array([Ck[0] for Ck in C] + [C[-1][0]])
    for k0 in range(0, n, seg):
        rng_ = np.max(np.abs(starts[k0:min(n, k0 + seg) + 1]), axis=0).reshape(m, N).max(1)
```

```
        sig = 1.0 / np.maximum(rng_, 1e-2)
        if S.spec.get("sigma_rule") == "unit": sig = np.ones(m)
        if S.spec.get("sigma_rule") == "global":
            sig = 1.0 / np.maximum(np.max(np.abs(starts), axis=0).reshape(m, N).max(1), 1e-2)
        sw = np.tile(np.sqrt(S.w), m) * np.repeat(sig, N)
        steps += [dict(sig=sig, sw=sw, swmin_n=swmin_n)] * (min(n, k0 + seg) - k0)
    s0 = steps[0]["sig"]; Dcap = cap_rel * max(1.0, float(np.max(np.abs(np.repeat(s0, N) * S.u0))))
    for cx in {id(c): c for c in steps}.values():
        cx["dcap"] = [Dcap / (cx["sig"][b] * swmin_n) for b in range(m)]
    return dict(steps=steps, Dcap=Dcap, sig=steps[-1]["sig"], sw=steps[0]["sw"])


class Cert:
    def __init__(s, S, C, q0=16, every=None, cap_rel=1e-2, q=None, e0_extra=0.0, e0_raw=None, hk=None):
        s.S, s.C = S, list(C); s.n = len(C)
        s.hk = np.full(s.n, S.T / s.n) if hk is None else np.asarray(hk, float)     # step sizes (variab
le steps allowed)
        s.tk = np.concatenate([[0.0], np.cumsum(s.hk)]); s.h = float(np.mean(s.hk)); s.ctx = make_ctx(S,
 C, cap_rel); s.cap_rel = cap_rel
        s.q = np.full(s.n, q0) if q is None else q.copy()
        s.d = [step_data(S, s.C, k, int(s.q[k]), s.ctx, s.hk[k]) for k in range(s.n)]
        if every is None:     # checkpoint spacing rule: a block spans at most one e-fold of the frozen
log-norm
            mu = max(max(float(x["mu"].max()) for x in s.d), 1e-12)
            every = int(min(100, max(5, math.floor(1.0 / (s.h * mu)))))
        s.every = every
        s.cps = list(range(every, s.n, every)) + [s.n]; s.starts = [0] + s.cps[:-1]
        s.blk = np.zeros(s.n, int)
        for i, (b0, c) in enumerate(zip(s.starts, s.cps)): s.blk[b0:c] = i
        if e0_raw is not None:            # componentwise bound |e_a(0)| <= r_a carried from a previous
segment
            e0_extra = float(np.linalg.norm(s.ctx["sw"] * np.repeat(np.asarray(e0_raw, float), S.N)))
        s.e0 = float(np.linalg.norm(s.ctx["sw"] * (C[0][0] - S.u0))) + e0_extra; s.e0_extra = e0_extra;
s.products()

    def redo(s, ks):
        for k in ks: s.d[k] = step_data(s.S, s.C, k, int(s.q[k]), s.ctx, s.hk[k])
        s._dirty = True; s.products()

    def products(s):
        n, dim = s.n, s.d[0]["E"].shape[0]; E = [x["E"] for x in s.d]; Ei = [x["Ei"] for x in s.d]
        s.nb, s.n0 = {}, {}
        eE = [x.get("eE", 0.0) for x in s.d]; eEi = [x.get("eEi", 0.0) for x in s.d]
        s.en = np.zeros(n)
        for k in range(n):
            hs = s.hk[k] / s.q[k]
            s.en[k] = min((pnorm_up(E[k]) + eE[k]) if RIG else pnorm(E[k]) * (1 + 1e-12),
                          math.exp(min(700, hs * float(np.sum(s.d[k]["mu"])))) * (PADF if RIG else 1.0))
        s.nb2 = {}
        for c in s.cps:
            # RIG (telescoping form): the chain uses the STORED products Phat_{c,k} = fl(Phat_{c,k+1} Eh
at_k) themselves.
            # With R_k = Phat_{c,k} - Phat_{c,k+1} Ehat_k (rounding) and e_{k+1} = Ehat_k e_k + (Phi_k -
 Ehat_k) e_k + f_k,
            #   e_c = Phat_{c,0} e_0 + sum_k Phat_{c,k+1} [f_k + (Phi_k - Ehat_k) e_k] - sum_k R_k e_k
  (exact identity),
            # so rounding enters only as level-proportional forcing (||R_k|| + eE_k) * Delta_k and never
 compounds.
            Pm = np.eye(dim); arr = np.zeros(c); arr2 = np.zeros(c)
            for k in range(c - 1, -1, -1):
                arr[k] = pnorm_up(Pm) if RIG else pnorm(Pm) * (1 + 1e-12)
                if RIG: arr2[k] = gam(dim + 2) * nb(Pm) * nb(E[k])
                Pm = Pm @ E[k]
            s.nb[c] = arr; s.nb2[c] = arr2; s.n0[c] = pnorm_up(Pm) if RIG else pnorm(Pm) * (1 + 1e-12)
        s.pre = np.ones(n + 1); s.pinv = np.ones(n + 1)
        for b0, c in zip(s.starts, s.cps):
            Pf = np.eye(dim); Pi = np.eye(dim); dF = 0.0; dI = 0.0
            for k in range(b0 + 1, c + 1):
                with np.errstate(over="ignore", invalid="ignore"):
                    if RIG:
                        Pf, dF = mul_rig(E[k - 1], eE[k - 1], Pf, dF, Na=s.en[k - 1]); s.pre[k] = pnorm_
up(Pf) + dF
                        Pi, dI = mul_rig(Pi, dI, Ei[k - 1], eEi[k - 1]); v = pnorm_up(Pi) + dI
                    else:
                        Pf = E[k - 1] @ Pf; s.pre[k] = pnorm(Pf) * (1 + 1e-12)
                        Pi = Pi @ Ei[k - 1]; v = pnorm(Pi) * (1 + 1e-8)
                    s.pinv[k] = v if np.isfinite(v) else np.inf


    def _arrays(s):
```

```
        qm = int(s.q.max()); n = s.n
        s.A_frm = np.zeros((n, qm)); s.A_hm = np.zeros((n, qm)); s.A_gn = np.zeros((n, qm)); s.A_mu = np.full((n, qm), -np.inf)
        for k, x in enumerate(s.d):
            qq = len(x["frm"]); s.A_frm[k, :qq] = x["frm"]; s.A_hm[k, :qq] = x["hm"]; s.A_gn[k, :qq] = x["gn"]; s.A_mu[k, :qq] = x["mu"]
        s.A_hs = s.hk / s.q.astype(float); s.A_mask = np.isfinite(s.A_mu)
        s.A_jump = np.array([x["jump"] for x in s.d]); s.A_conv = np.array([x.get("conv", 1.0) for x in s.d])
        s.A_eE = np.array([x.get("eE", 0.0) for x in s.d])          # ||Phi_k - Ehat_k||: level-proportional forcing

    def ell(s, Dk):
        if not hasattr(s, "A_frm") or s.A_frm.shape[1] != int(s.q.max()) or getattr(s, "_dirty", True): s._arrays(); s._dirty = False
        D = np.asarray(Dk, float)[:, None]; hs = s.A_hs[:, None]; M = s.A_mask
        with np.errstate(over="ignore", invalid="ignore"):
            eta = s.A_frm + s.A_hm * D; mu = np.where(M, s.A_mu, 0.0)
            w = np.where(M, hs * np.exp(np.minimum(hs * (mu + eta), 700)), 0.0)
            grow = np.exp(np.minimum(s.hk * (np.max(np.where(M, s.A_mu, -np.inf), 1) + np.max(np.where(M, eta, -np.inf), 1)), 700))
            a_g = np.sum(w * s.A_gn, 1); a_f = np.sum(w * s.A_frm, 1) * D[:, 0]; a_h = np.sum(w * s.A_hm, 1) * D[:, 0] ** 2
        loc = grow * (a_g + a_f + a_h); cv = s.A_conv
        P4 = np.column_stack([cv * grow * a_g, cv * grow * a_f + (s.A_eE * D[:, 0] if RIG else 0.0), cv * grow * a_h, s.A_jump])
        s._loc = loc
        return P4.sum(1), grow, P4

    def level_block(s, Dl, i):
        ell, grow, _ = s.ell(Dl[s.blk])
        Dk = np.asarray(Dl, float)[s.blk]
        Bc = {c: float(np.dot(s.nb[c], ell[:c]) + s.n0[c] * s.e0 + (np.dot(s.nb2[c], Dk[:c]) if RIG else 0.0)) * PADF for c in s.cps[:i + 1]}
        b0, c = s.starts[i], s.cps[i]; Bs = s.e0 if b0 == 0 else Bc[b0]; mx = Bc[c]
        cl = np.concatenate([[0.0], np.cumsum(np.log(s.en[b0:c]))])
        for k in range(b0, c):
            if k == b0: Ik = Bs
            else:
                js = np.arange(b0, k)
                w1 = np.exp(np.minimum(cl[k - b0] - cl[js + 1 - b0], 700)); w2 = s.pre[k] * s.pinv[js + 1]
                W = np.minimum(w1, w2); W[js == k - 1] = 1.0
                Ik = s.pre[k] * Bs + float(np.dot(W, ell[js]))
            mx = max(mx, (grow[k] * Ik + max(s._loc[k], ell[k])) * PADF)
        return mx, Bc

    def close(s):
        """causal prefix closure: block i only depends on blocks <= i, so each block is closed once, in order"""
        s._dirty = True
        nbk = len(s.cps); Dl = np.zeros(nbk); Dcap = s.ctx["Dcap"]; good = 0; Bfin = {}
        for i in range(nbk):
            x = 0.0; ok = False
            for _ in range(200):
                Dl[i] = x; y, _ = s.level_block(Dl, i)
                if not np.isfinite(y) or y > Dcap: break
                if abs(y - x) <= 1e-6 * y: ok = True; break
                x = y
            if ok:
                Dl[i] = 1.02 * y + 1e-300; y2, Bc = s.level_block(Dl, i)
                ok = bool(y2 <= Dl[i] and Dl[i] <= Dcap)
            if not ok: Dl[i] = Dcap; break
            good = i + 1; Bfin = Bc
        return good, Dl, Bfin

    def attribute(s, Dl, i):
        c = s.cps[i]; _, _, P4 = s.ell(Dl[s.blk]); return s.nb[c][:c, None] * P4[:c]

    def localize(s, Dl, i, top=8):
        """where and how: the chain-weighted share of the bound at checkpoint i, split exactly by step and by term
        (defect, freezing, quadratic, jump); each step's term is then placed on (component, node):
          defect, jump  -> split in proportion to the squared weighted entries (the shares add up to the term),
          freezing, quadratic -> the node where the sub-step maximum is attained.
        Returns the share map (n_steps, 4, m, N) and the top entries (step range, term, component, node, share)."""
        m, N = s.S.m, s.S.N; con = s.attribute(Dl, i); c = con.shape[0]; names = ["defect", "freezing", "quadratic", "jump"]
```

```
        M = np.zeros((c, 4, (m + 1) * N))          # component m = "all components at this node" (freezing, quadratic)
        for k in range(c):
            wh = s.d[k].get("where")
            if wh is None: continue
            for t, key in enumerate(names):
                v = np.asarray(wh[key], float)
                if key in ("defect", "jump"):
                    tot = v.sum(); M[k, t, :m * N] = con[k, t] * (v / tot if tot > 0 else np.full(v.size, 1.0 / v.size))
                else:
                    j = int(np.argmax(v)); M[k, t, (m * N + j) if v.size == N else j] = con[k, t]
        M = M.reshape(c, 4, m + 1, N); total = float(con.sum())
        nz = np.argwhere(M > 0); flat = sorted(((float(M[tuple(ix)]), *map(int, ix)) for ix in nz), reverse=True)
        flat.sort(reverse=True)
        rows = [dict(t=[float(s.tk[k]), float(s.tk[k + 1])], term=names[t], component=("all" if a == m else a), node=j, share=v / total)
                for v, k, t, a, j in flat[:top]]
        return M, rows


def taylor_from(S, y, h, K=8):
    m, N = S.m, S.N; c = [y.copy()]
    for j in range(K):
        Ps = [P(np.array(c)[:, a * N:(a + 1) * N]) for a in range(m)]
        r = [P.k(v, N) for v in S.R_gen(*Ps)]
        c.append(np.concatenate([(S.D[a] * (S.L @ c[j][a * N:(a + 1) * N]) + (r[a].c[j] if len(r[a].c) > j else 0)
                                  + (S.src[a * N:(a + 1) * N] if j == 0 else 0)) / (j + 1) for a in range(m)]))
    return np.array(c)


def certify(S, C, rounds=8, frac=0.9, repair=True, log=None, e0_extra=0.0, cap_rel=1e-2, e0_raw=None, hk=None):
    """returns the history of rounds; each round is a boolean certificate on [0, t*] plus the attribution of what blocks t*"""
    t0 = time.time(); C = list(C)
    bad = [k for k, Ck in enumerate(C) if not np.all(np.isfinite(Ck)) or np.max(np.abs(Ck[0])) > 1e8]
    if bad:                                                  # the candidate itself diverged: nothing to certify
        rec = dict(round=0, proved_full=False, t_star=0.0, eps=None, bounds=[], levels=[], dominant="candidate diverged",
                   t_range90=[0.0, 0.0], action="stop", wall_s=time.time() - t0)
        return [rec], rec
    cert = Cert(S, C, e0_extra=e0_extra, cap_rel=cap_rel, e0_raw=e0_raw, hk=hk); hist = []; rep = set(); best = None
    for r in range(rounds + 1):
        good, Dl, Bc = cert.close(); nbk = len(cert.cps); tstar = float(cert.tk[cert.cps[good - 1]]) if good else 0.0
        rec = dict(round=r, proved_full=good == nbk, t_star=tstar, eps=Bc.get(cert.cps[good - 1]) if good else None,
                   bounds=[[float(cert.tk[cert.cps[i]]), Bc[cert.cps[i]]] for i in range(good)], cap_rel=cert.cap_rel,
                   levels=[[float(cert.tk[cert.starts[i]]), float(cert.tk[cert.cps[i]]), float(Dl[i])] for i in range(good)], sigma=cert.ctx["sig"].tolist(),
                   sigma_blocks=[np.min([cert.ctx["steps"][k]["sig"] for k in range(cert.starts[i], cert.cps[i])], axis=0).tolist() for i in range(good)],
                   repaired=len(rep), substeps=int(cert.q.sum()), q_max=int(cert.q.max()), wall_s=time.time() - t0)
        act = None
        try:
            _, rec["where"] = cert.localize(Dl, min(good, nbk - 1))
        except Exception as ex:
            rec["where"] = str(ex)
        if good < nbk:
            con = cert.attribute(Dl, good); tot = con.sum(0); names = ["defect", "freezing", "quadratic", "jump"]
            dom = names[int(np.argmax(tot))]; per = con.sum(1); order = np.argsort(-per); cum = np.cumsum(per[order]) / per.sum()
            ks = order[:int(np.searchsorted(cum, frac)) + 1]
            act = {"quadratic": "cap/10", "freezing": "q*4"}.get(dom, "local Taylor" if (repair and dom == "defect") else "stop")
            new_cap = cert.cap_rel / 10
            if act == "cap/10" and good:
                # the cap may not drop below the level already reached: smallest cap safely above it
                lev = float(np.max(Dl[:good])) / (cert.ctx["Dcap"] / cert.cap_rel)
                new_cap = max(cert.cap_rel / 10, 2.5 * lev)
                if new_cap >= 0.9 * cert.cap_rel:          # no room: lower the base error instead
```